Berachain BeaconKit (Polaris) 상세 분석

작성일: 2025-01-07 **분석 대상**: BeaconKit (beacon-kit repository) **버전**: Latest (2025년 1월 기준)

목차

1. 개요
2. BeaconKit 아키텍처
3. Engine API 통합
4. State Transition 구조
5. ABCI 구현
6. 실행 엔진 연동
7. Payload 관리
8. 검증 로직
9. Omni/MiniEVM과의 비교
10. 성능 및 최적화
11. 장단점 분석
12. 실전 예제

---

1. 개요

1.1 BeaconKit이란?

BeaconKit은 **Berachain이 개발한 모듈러 프레임워크**로, EVM 기반 합의 클라이언트(consensus client)를 구축하기 위한 도구입니다.

```
// beacon-kit/README.md에서 확인한 실제 정의:
// "A modular framework for building EVM consensus clients"
```

핵심 특징:

- Ethereum Beacon Chain 스타일의 합의 레이어
- Ethereum Engine API를 통한 실행 레이어 분리
- CometBFT 기반 합의 메커니즘
- Polaris라고도 불리는 EVM 통합 레이어

1.2 지원 실행 클라이언트

BeaconKit은 Engine API를 통해 다음 실행 클라이언트를 지원합니다:

1. **Bera-Geth**: Go 언어 기반, Ethereum Geth의 Berachain 포크
2. **Bera-Reth**: Rust 언어 기반, 성능 최적화 중점

```go
// beacon-kit/execution/client/client.go (lines 38-55)
// 실제 EngineClient 구조체 정의

type EngineClient struct {
    *ethclient.Client
    // cfg is the supplied configuration for the engine client.
    cfg *Config
    // logger is the logger for the engine client.
    logger log.Logger
    // eth1ChainID is the chain ID of the execution client.
    eth1ChainID *big.Int
    // clientMetrics is the metrics for the engine client.
    metrics *clientMetrics
    // capabilities is a map of capabilities that the execution client has.
    capabilities map[string]struct{}
    // connected will be set to true when we have successfully connected
    // to the execution client.
    connectedMu sync.RWMutex
    connected   bool
}
```

---

2. BeaconKit 아키텍처

2.1 전체 아키텍처

```
┌─────────────────────────────────────────────────────────────┐
│                      BeaconKit Framework                    │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  ┌──────────────┐          ┌──────────────┐                │
│  │   CometBFT   │◄────────►│  ABCI Service │                │
│  │   Consensus  │          │  (service.go) │                │
│  └──────────────┘          └──────────────┘                │
│        │                         │                          │
│        │                         ▼                          │
│        │                  ┌──────────────┐                  │
│        │                  │ StateProcessor │                │
│        │                  │  (core/)     │                  │
│        │                  └──────────────┘                  │
│        │                         │                          │
│        ▼                         ▼                          │
│  ┌──────────────────────────────────────┐                   │
│  │        Blockchain Service            │                   │
│  │  (beacon/blockchain/service.go)      │                   │
│  └──────────────────────────────────────┘                   │
│                   │                                         │
│                   ▼                                         │
│         ┌──────────────────┐                                │
│         │  EngineClient    │                                │
│         │  (execution/client) │                             │
│         └──────────────────┘                                │
│                   │                                         │
└───────────────────┼─────────────────────────────────────────┘
                    │ Engine API (JSON-RPC)
                    ▼
          ┌──────────────────┐
          │ Execution Client │
          │ (Bera-Geth/Reth) │
          └──────────────────┘
```

2.2 주요 컴포넌트

2.2.1 CometBFT Service

```go
// beacon-kit/consensus/cometbft/service/service.go (lines 52-100)
// 실제 Service 구조체 정의

type Service struct {
    node        *node.Node
    nodeAddress cmtcrypto.Address

    delayCfg delay.ConfigGetter

    // cmtConsensusParams are part of the blockchain state and
    // are agreed upon by all validators in the network.
    cmtConsensusParams *cmttypes.ConsensusParams

    // cmtCfg are node-specific settings that influence how
    // the consensus engine operates on a particular node.
    // Loaded from config.toml, not part of state.
    cmtCfg *cmtcfg.Config

    telemetrySink TelemetrySink

    logger       *phuslu.Logger
    sm           *statem.Manager
    Blockchain   blockchain.BlockchainI
    BlockBuilder validator.BlockBuilderI

    // cachedStates tracks in memory the post block states
    // of blocks which were successfully verified. It allows
    // finalizing without re-execution
    cachedStates cache.States

    interBlockCache storetypes.MultiStorePersistentCache

    // initialHeight is the initial height at which we start the node
    initialHeight   int64
    minRetainBlocks uint64

    chainID string

    // ctx is the context passed in for the service
    ctx context.Context
}
```

특징:

- CometBFT 노드를 래핑하여 ABCI 인터페이스 제공
- 상태 캐싱을 통한 재실행 방지
- 블록 빌더와 블록체인 서비스 통합

2.2.2 StateProcessor

```go
// beacon-kit/state-transition/core/state_processor.go (lines 40-82)
// 실제 StateProcessor 구조체

type StateProcessor struct {
    // logger is used for logging information and errors.
    logger log.Logger
    // cs is the chain specification for the beacon chain.
    cs ChainSpec
    // signer is the BLS signer used for cryptographic operations.
    signer crypto.BLSSigner
    // fGetAddressFromPubKey verifies that a validator public key
    // matches with the proposer address passed by the consensus
    // Injected via ctor to simplify testing.
    fGetAddressFromPubKey func(crypto.BLSPubkey) ([]byte, error)
    // executionEngine is the engine responsible for executing transactions.
    executionEngine ExecutionEngine
    // ds allows checking payload deposits against the deposit contract
    ds deposit.StoreManager
    // metrics is the metrics for the service.
    metrics *stateProcessorMetrics
    // logDeneb1Once enforces logging the Deneb1 fork information at most once.
    logDeneb1Once sync.Once
}

// NewStateProcessor creates a new state processor.
func NewStateProcessor(
    logger log.Logger,
    cs ChainSpec,
    executionEngine ExecutionEngine,
    ds deposit.StoreManager,
    signer crypto.BLSSigner,
    fGetAddressFromPubKey func(crypto.BLSPubkey) ([]byte, error),
    telemetrySink TelemetrySink,
) *StateProcessor {
    return &StateProcessor{
        logger:                logger,
        cs:                    cs,
        executionEngine:       executionEngine,
        signer:                signer,
        fGetAddressFromPubKey: fGetAddressFromPubKey,
        ds:                    ds,
        metrics:               newStateProcessorMetrics(telemetrySink),
    }
}
```

주요 역할:

- 비콘 체인의 상태 전이 처리
- 실행 엔진과의 상호작용 조율
- BLS 서명 검증
- 디포짓 검증

---

3. Engine API 통합

3.1 Engine API 개요

BeaconKit은 **Ethereum Engine API** 표준을 구현하여 합의 레이어와 실행 레이어를 분리합니다.

Engine API 주요 메서드:

1. `engine_newPayloadVX` : 새 실행 페이로드 제출
2. `engine_forkchoiceUpdatedVX` : 포크 선택 상태 업데이트
3. `engine_getPayloadVX` : 빌드된 페이로드 가져오기

3.2 EngineClient 구현

3.2.1 NewPayload

```go
// beacon-kit/execution/client/engine.go (lines 39-74)
// 실제 NewPayload 구현

// NewPayload calls the engine_newPayloadVX method via JSON-RPC.
func (s *EngineClient) NewPayload(
    ctx context.Context,
    req ctypes.NewPayloadRequest,
) (*common.ExecutionHash, error) {
    var (
        startTime    = time.Now()
        cctx, cancel = s.createContextWithTimeout(ctx)
    )
    defer s.metrics.measureNewPayloadDuration(startTime)
    defer cancel()

    // Call the appropriate RPC method based on the payload version.
    result, err := s.Client.NewPayload(cctx, req)
    if err != nil {
        if errors.Is(err, engineerrors.ErrEngineAPITimeout) {
            s.metrics.incrementNewPayloadTimeout()
        }
        return nil, s.handleRPCError(err)
    }
    if result == nil {
        return nil, engineerrors.ErrNilPayloadStatus
    }

    // This case is only true when the payload is invalid, so
    // `processPayloadStatusResult` below will return an error.
    if validationErr := result.ValidationError; validationErr != nil {
        s.logger.Error(
            "Got a validation error in newPayload",
            "err",
            errors.New(*validationErr),
        )
    }

    return processPayloadStatusResult(result)
}
```

특징:

- 타임아웃 관리
- 메트릭 수집
- 에러 처리 및 로깅
- 페이로드 버전별 처리

3.2.2 ForkchoiceUpdated

```go
// beacon-kit/execution/client/engine.go (lines 80-119)
// 실제 ForkchoiceUpdated 구현

// ForkchoiceUpdated calls the engine_forkchoiceUpdatedV1 method via JSON-RPC.
func (s *EngineClient) ForkchoiceUpdated(
    ctx context.Context,
    state *engineprimitives.ForkchoiceStateV1,
    attrs *engineprimitives.PayloadAttributes,
    forkVersion common.Version,
) (*engineprimitives.PayloadID, error) {
    var (
        startTime    = time.Now()
        cctx, cancel = s.createContextWithTimeout(ctx)
    )
    defer s.metrics.measureForkchoiceUpdateDuration(startTime)
    defer cancel()

    // If the suggested fee recipient is not set, log a warning.
    if attrs != nil &&
        attrs.GetSuggestedFeeRecipient() == (common.ExecutionAddress{}) {
        s.logger.Warn(
            "Suggested fee recipient is not configured 🔆",
            "fee-recipent", attrs.GetSuggestedFeeRecipient(),
        )
    }

    result, err := s.Client.ForkchoiceUpdated(cctx, state, attrs, forkVersion)
    if err != nil {
        if errors.Is(err, engineerrors.ErrEngineAPITimeout) {
            s.metrics.incrementForkchoiceUpdateTimeout()
        }
        return nil, s.handleRPCError(err)
    }
    if result == nil {
        return nil, engineerrors.ErrNilForkchoiceResponse
    }

    _, err = processPayloadStatusResult(&result.PayloadStatus)
    if err != nil {
        return nil, err
    }
    return result.PayloadID, nil
}
```

특징:

- 포크 선택 상태 전송
- 페이로드 속성 전달 (옵셔널)
- Fee recipient 검증
- PayloadID 반환

3.2.3 GetPayload

```go
// beacon-kit/execution/client/engine.go (lines 125-150)
// 실제 GetPayload 구현

// GetPayload calls the engine_getPayloadVX method via JSON-RPC. It returns
// the execution data as well as the blobs bundle.
func (s *EngineClient) GetPayload(
    ctx context.Context,
    payloadID engineprimitives.PayloadID,
    forkVersion common.Version,
) (ctypes.BuiltExecutionPayloadEnv, error) {
    var (
        startTime    = time.Now()
        cctx, cancel = s.createContextWithTimeout(ctx)
    )
    defer s.metrics.measureGetPayloadDuration(startTime)
    defer cancel()

    // Call and check for errors.
    result, err := s.Client.GetPayload(cctx, payloadID, forkVersion)
    if err != nil {
        if errors.Is(err, engineerrors.ErrEngineAPITimeout) {
            s.metrics.incrementGetPayloadTimeout()
        }
        return result, s.handleRPCError(err)
    }
    if result == nil {
        return result, engineerrors.ErrNilExecutionPayloadEnvelope
    }
    if result.GetBlobsBundle() == nil {
        // ... blob 검증 로직
```

특징:

- PayloadID로 빌드된 페이로드 조회
- Blob 번들 검증
- 실행 페이로드 반환

3.3 클라이언트 시작 및 연결

```go
// beacon-kit/execution/client/client.go (lines 104-150)
// 실제 Start 메서드

// Start the engine client.
func (s *EngineClient) Start(ctx context.Context) error {
    // Start the Client.
    go s.Client.Start(ctx)

    s.logger.Info(
        "Initializing connection to the execution client...",
        "dial_url", s.cfg.RPCDialURL.String(),
    )

    // If the connection connection succeeds, we can skip the
    // connection initialization loop.
    if err := s.verifyChainIDAndConnection(ctx); err == nil {
        return nil
    }

    // Attempt to initialize the connection to the execution client.
    ticker := time.NewTicker(s.cfg.RPCStartupCheckInterval)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-ticker.C:
            s.logger.Info(
                "Waiting for execution client to start... 🍺🕓",
                "dial_url", s.cfg.RPCDialURL,
            )
            if err := s.verifyChainIDAndConnection(ctx); err != nil {
                if errors.Is(err, ErrMismatchedEth1ChainID) {
                    s.logger.Error(err.Error())
                }
                continue
            }
            s.connectedMu.Lock()
            s.connected = true
            s.connectedMu.Unlock()
            return nil
        }
    }
}
```

특징:

- 비동기 클라이언트 시작
- 연결 재시도 로직
- ChainID 검증
- 연결 상태 관리

---

4. State Transition 구조

4.1 Transition 메서드

```go
// beacon-kit/state-transition/core/state_processor.go (lines 84-119)
// 실제 Transition 구현

// Transition is the main function for processing a state transition.
func (sp *StateProcessor) Transition(
    ctx ReadOnlyContext,
    st *state.StateDB,
    blk *ctypes.BeaconBlock,
) (transition.ValidatorUpdates, error) {
    if blk == nil {
        return nil, nil
    }

    // Process the next slot.
    validatorUpdates, err := sp.ProcessSlots(st, blk.GetSlot())
    if err != nil {
        return nil, err
    }

    // Prepare the state for the next block's fork version.
    // Ideally we want to log only in case we are processing the
    // block to be finalized. Pre cache activation this is easy.
    // Post activation we log every time we verify a block
    logForkProcessing := ctx.VerifyPayload() && !ctx.VerifyRandao()

    if cache.IsStateCachingActive(sp.cs, blk.Slot) {
        logForkProcessing = ctx.VerifyPayload()
    }
    if err = sp.ProcessFork(st, blk.GetTimestamp(), logForkProcessing); err != nil {
        return nil, err
    }

    // Process the block.
    if err = sp.ProcessBlock(ctx, st, blk); err != nil {
        return nil, err
    }

    return validatorUpdates, nil
}
```

상태 전이 단계:

1. **ProcessSlots**: 슬롯 처리 (에포크 전이 포함)
2. **ProcessFork**: 포크 버전 업데이트
3. **ProcessBlock**: 블록 처리 (페이로드 검증 포함)

4.2 ProcessSlots

```go
// beacon-kit/state-transition/core/state_processor.go (lines 121-150)
// 실제 ProcessSlots 구현 주석

// ProcessSlots deviates from the ethereum consensus specs `process_slots`.
// The `process_slots` function must iterate and process slots in which the
// target `slot` can be several slots ahead of the `stateSlot`. This is because
// the beacon chain can miss blocks for a given slot, resulting in not processing
// the slot. For example, the current slot is 100, and no blocks were proposed
// for slots 101 and 102. Upon receiving a block at slot 103, the beacon state
// must "catch up" the state and trigger slot and epoch transitions that may
// have happened during slot 101 and 102.
//
// Beacon-kit does not allow missed slots. Each height from cometBFT will always
// correspond to a beacon block slot, so `ProcessSlots` will always be called at
// every slot. Thus, we will only process the state up to the next slot.
// The reasoning behind this deviation is to be explicit in this behavior and
// also to better support the usage of fork logic in the `processEpoch` function.
// Since we do not fork by slot but instead fork by timestamp, we must be able to
// strictly tie each call of `processSlot` and `processEpoch` to a timestamp.
// Since we don't have beacon blocks during each iteration of the slot loop,
// we cannot correlate each slot to a timestamp. We instead identify that we
// process only one slot, allowing us to simply use the fork version from the state.
func (sp *StateProcessor) ProcessSlots(
    st *state.StateDB, slot math.Slot,
) (transition.ValidatorUpdates, error) {
    var res transition.ValidatorUpdates

    stateSlot, err := st.GetSlot()
    if err != nil {
        return nil, err
    }
    if slot == stateSlot {
        return res, nil
    }
    if slot != stateSlot+1 {
        return nil, fmt.Errorf("slot %d does not match expected slot %d",
            slot, stateSlot+1)
    }

    if err = sp.processSlot(st); err != nil {
        // ... 에러 처리
```

BeaconKit의 특징:

- **Missed slots 불허**: CometBFT의 모든 높이는 비콘 블록 슬롯에 대응
- **순차 처리**: 슬롯을 하나씩만 처리 (catch-up 없음)
- **타임스탬프 기반 포크**: 슬롯이 아닌 타임스탬프로 포크 결정

---

5. ABCI 구현

5.1 PrepareProposal

```go
// beacon-kit/consensus/cometbft/service/prepare_proposal.go (lines 33-90)
// 실제 PrepareProposal 구현

func (s *Service) prepareProposal(
    ctx context.Context,
    req *cmtabci.PrepareProposalRequest,
) (*cmtabci.PrepareProposalResponse, error) {
    startTime := time.Now()
    defer s.telemetrySink.MeasureSince(
        "beacon_kit.runtime.prepare_proposal_duration", startTime)

    // CometBFT must never call PrepareProposal with a height of 0.
    if req.Height < 1 {
        return nil, fmt.Errorf(
            "prepareProposal at height %v: %w",
            req.Height,
            errInvalidHeight,
        )
    }

    // prepareProposalState is used for PrepareProposal, which is set based on
    // the previous block's state. This state is never committed. In case of
    // multiple consensus rounds, the state is always reset to the previous
    // block's state. Always reset state given that PrepareProposal can timeout
    // and be called again in a subsequent round.
    prepareProposalState := s.resetState(ctx)
    //nolint:contextcheck // ctx already passed via resetState
    prepareProposalState.SetContext(
        s.getContextForProposal(
            prepareProposalState.Context(),
            req.Height,
        ),
    )

    slotData := types.NewSlotData(
        math.Slot(req.GetHeight()), // #nosec G115
        nil,                        // no attestations
        nil,                        // no slashings
        req.GetProposerAddress(),
        req.GetTime(),
    )

    //nolint:contextcheck // ctx already passed via resetState
    blkBz, sidecarsBz, err := s.BlockBuilder.BuildBlockAndSidecars(
        prepareProposalState.Context(),
        slotData,
    )
    if err != nil {
        s.logger.Error(
            "failed to prepare proposal",
            "height", req.Height,
            "time", req.Time,
            "err", err,
        )
        return &cmtabci.PrepareProposalResponse{Txs: [][]byte{}}, nil
    }

    return &cmtabci.PrepareProposalResponse{
        Txs: [][]byte{blkBz, sidecarsBz},
    }, nil
}
```

특징:

- 이전 블록 상태를 기반으로 상태 리셋
- 타임아웃 시 재호출 대비
- 블록과 사이드카를 별도 트랜잭션으로 반환
- 에러 시 빈 제안 반환

5.2 FinalizeBlock

```go
// beacon-kit/consensus/cometbft/service/finalize_block.go (lines 38-131)
// 실제 FinalizeBlock 구현 (일부)

func (s *Service) finalizeBlock(
    ctx context.Context,
    req *cmtabci.FinalizeBlockRequest,
) (*cmtabci.FinalizeBlockResponse, error) {
    if err := s.validateFinalizeBlockHeight(req); err != nil {
        return nil, err
    }

    // Check whether currently block hash is already available. If so
    // we may speed up block finalization.
    hash := string(req.Hash)
    switch cached, err := s.cachedStates.GetCached(hash); {
    case err == nil:
        // Block with height equal to initial height is special and we can't rely on its cache.
        // This is because Genesis state is cached but not committed (and purged from s.cachedStates)
        // We handle the case outside of this switch, via the s.Blockchain.FinalizeBlock below.
        if req.Height > s.initialHeight {
            if err = s.cachedStates.MarkAsFinal(hash); err != nil {
                return nil, fmt.Errorf("failed marking state as final, hash %s, height %d: %w",
                    hash, req.Height, err)
            }

            finalState := cached.State
            var (
                signedBlk *ctypes.SignedBeaconBlock
                sidecars  datypes.BlobSidecars
            )
            signedBlk, sidecars, err = s.Blockchain.ParseBeaconBlock(req)
            if err != nil {
                return nil, fmt.Errorf("finalize block: failed parsing block: %w", err)
            }
            blk := signedBlk.GetBeaconBlock()

            if err = s.Blockchain.FinalizeSidecars(
                finalState.Context(),
                req.SyncingToHeight,
                blk,
                sidecars,
            ); err != nil {
                return nil, fmt.Errorf("failed finalizing sidecars: %w", err)
            }
            if err = s.Blockchain.PostFinalizeBlockOps(
                finalState.Context(),
                blk,
            ); err != nil {
                return nil, fmt.Errorf("finalize block: failed post finalize block ops: %w", err)
            }
            return s.calculateFinalizeBlockResponse(req, cached.ValUpdates)
        }

    case errors.Is(err, cache.ErrStateNotFound):
        // this is a benign error, it just signal it's the first time we see the block being finalized
        // Keep processing below

    default:
        return nil, fmt.Errorf("failed checking cached state, hash %s, height %d: %w",
            hash, req.Height, err)
    }

    // Block has not been cached already, as it happens when we block sync.
    // Normally we would directly process it but first block post initial height
    // needs special handling.
    var finalState *cache.State
    cachedFinalHash, cachedFinalState, err := s.cachedStates.GetFinal()
    switch {
    case errors.Is(err, cache.ErrNoFinalState):
        hash = string(req.Hash) // not necessary but clarifies intent
        finalState = s.resetState(ctx)

    case err == nil:
        hash = cachedFinalHash
        finalState = cachedFinalState
```

```go
        // Preserve the CosmosSDK context while using the correct base ctx.
        finalState.SetContext(finalState.Context().WithContext(ctx))

    default:
        return nil, fmt.Errorf("failed checking cached final state, hash %s, height %d: %w",
            hash, req.Height, err)
    }

    valUpdates, err := s.Blockchain.FinalizeBlock(finalState.Context(), req)
    if err != nil {
        return nil, err
    }

    // Cache and mark state as final
    s.cachedStates.SetCached(hash, &cache.Element{
        State:      finalState,
        ValUpdates: valUpdates,
    })
    if err = s.cachedStates.MarkAsFinal(hash); err != nil {
        return nil, fmt.Errorf("failed marking state as final, hash %s, height %d: %w",
            hash, req.Height, err)
    }

    return s.calculateFinalizeBlockResponse(req, valUpdates)
}
```

주요 최적화:

1. **캐시 활용**: 이미 검증된 블록은 재실행 없이 최종화
2. **Genesis 특수 처리**: 초기 높이 블록은 캐시 사용 안 함
3. **상태 캐싱**: 검증 완료된 상태를 메모리에 보관
4. **Validator 업데이트**: 검증자 변경사항 반환

---

6. 실행 엔진 연동

6.1 Payload 검증

```go
// beacon-kit/state-transition/core/state_processor_payload.go (lines 36-103)
// 실제 processExecutionPayload 구현

// processExecutionPayload processes the execution payload and ensures it
// matches the local state.
func (sp *StateProcessor) processExecutionPayload(
    txCtx ReadOnlyContext,
    st *statedb.StateDB,
    blk *ctypes.BeaconBlock,
    parentProposerPubkey *crypto.BLSPubkey,
) error {
    var (
        body    = blk.GetBody()
        payload = body.GetExecutionPayload()
        header  = &ctypes.ExecutionPayloadHeader{} // appeases nilaway
        g, ctx  = errgroup.WithContext(txCtx.ConsensusCtx())
    )

    payloadTimestamp := payload.GetTimestamp().Unwrap()
    consensusTimestamp := txCtx.ConsensusTime().Unwrap()

    sp.metrics.gaugeTimestamps(payloadTimestamp, consensusTimestamp)

    sp.logger.Info("processExecutionPayload",
        "consensus height", blk.GetSlot().Unwrap(),
        "payload height", payload.GetNumber().Unwrap(),
        "payload timestamp", payloadTimestamp,
        "consensus timestamp", consensusTimestamp,
        "verify payload", txCtx.VerifyPayload(),
    )

    if version.EqualsOrIsAfter(blk.GetForkVersion(), version.Electra()) {
        requests, getErr := blk.GetBody().GetExecutionRequests()
        if getErr != nil {
            return getErr
        }
        sp.logger.Info(
            "Processing execution requests",
            "deposits", len(requests.Deposits),
            "withdrawals", len(requests.Withdrawals),
            "consolidations", len(requests.Consolidations),
        )
    }

    // Perform payload verification only if the context is configured as such.
    if txCtx.VerifyPayload() {
        g.Go(func() error {
            return sp.validateExecutionPayload(ctx, txCtx.ConsensusTime(),
                st, blk, parentProposerPubkey)
        })
    }

    // Get the execution payload header.
    g.Go(func() error {
        var err error
        header, err = payload.ToHeader()
        return err
    })

    if err := g.Wait(); err != nil {
        return err
    }

    if txCtx.MeterGas() {
        sp.metrics.gaugeBlockGasUsed(
            payload.GetNumber(), payload.GetGasUsed(), payload.GetBlobGasUsed(),
        )
    }

    // Set the latest execution payload header.
```

```
    return st.SetLatestExecutionPayloadHeader(header)
}
```

검증 단계:

1. **타임스탬프 검증**: 페이로드와 합의 타임스탬프 비교
2. **실행 요청 처리**: Electra 포크 이후 디포짓/출금/통합 처리
3. **병렬 검증**: errgroup으로 페이로드 검증과 헤더 생성 병렬화
4. **가스 메트릭**: 가스 사용량 측정
5. **상태 업데이트**: 최신 실행 페이로드 헤더 저장

6.2 Stateful 검증

```go
// beacon-kit/state-transition/core/state_processor_payload.go (lines 140-214)
// 실제 validateStatefulPayload 구현 (일부)

// validateStatefulPayload performs stateful checks on the execution payload.
func (sp *StateProcessor) validateStatefulPayload(
    ctx context.Context,
    consensusTime math.U64,
    st ReadOnlyBeaconState,
    blk *ctypes.BeaconBlock,
    parentProposerPubkey *crypto.BLSPubkey,
) error {
    body := blk.GetBody()
    payload := body.GetExecutionPayload()

    lph, err := st.GetLatestExecutionPayloadHeader()
    if err != nil {
        return err
    }

    // Check chain canonicity
    safeHash := lph.GetBlockHash()
    if safeHash != payload.GetParentHash() {
        return errors.Wrapf(
            ErrParentPayloadHashMismatch,
            "parent block with hash %x is not finalized, expected finalized hash %x",
            payload.GetParentHash(),
            safeHash,
        )
    }

    // Verify that the payload stamp is within a reasonable bound
    if err = payloadtime.Verify(
        consensusTime,
        lph.GetTimestamp(),
        payload.GetTimestamp(),
    ); err != nil {
        return err
    }

    payloadReq, err := ctypes.BuildNewPayloadRequestFromFork(blk, parentProposerPubkey)
    if err != nil {
        return err
    }

    // First we verify the block hash and versioned hashes are valid.
    // TODO: is this required? Or will the EL handle this for us during
    // new payload?
    if err = payloadReq.HasValidVersionedAndBlockHashes(); err != nil {
        return err
    }

    // TODO: set retryOnSyncingStatus to false if we are in FinalizeBlock.
    // Otherwise leave as true. This is ok to leave this way for now.
    if err = sp.executionEngine.NotifyNewPayload(ctx, payloadReq, true); err != nil {
        return err
    }

    // Verify RANDAO
    epoch, err := st.GetEpoch()
    if err != nil {
        return err
    }

    expectedMix, err := st.GetRandaoMixAtIndex(epoch.Unwrap() % sp.cs.EpochsPerHistoricalVector())
    if err != nil {
        return err
    }

    if payload.GetPrevRandao() != expectedMix {
        return errors.Wrapf(
            ErrRandaoMixMismatch,
            "prev randao does not match, expected: %x, got: %x",
            expectedMix, payload.GetPrevRandao(),
        )
    }
```

```
    return nil
}
```

검증 항목:

1. **체인 정합성**: 부모 해시 검증
2. **타임스탬프 범위**: 합리적 범위 내 타임스탬프 확인
3. **블록 해시**: 블록 해시 및 버전 해시 검증
4. **실행 엔진 통지**: NewPayload 호출로 실행 레이어 검증
5. **RANDAO 검증**: RANDAO mix 일치 확인

---

7. Payload 관리

7.1 ForkchoiceUpdate 전송

```go
// beacon-kit/beacon/blockchain/execution_engine.go (lines 32-62)
// 실제 sendPostBlockFCU 구현

// sendPostBlockFCU sends a forkchoice update to the execution client after a
// block is finalized. This function should only be used to notify
// the EL client of the new head and should not request optimistic builds, as:
// Optimistic clients already request builds in handleOptimisticPayloadBuild()
// Non-optimistic clients should never request optimistic builds.
func (s *Service) sendPostBlockFCU(
    ctx context.Context,
    st *statedb.StateDB,
) error {
    lph, err := st.GetLatestExecutionPayloadHeader()
    if err != nil {
        return fmt.Errorf("failed getting latest payload: %w", err)
    }

    // Send a forkchoice update without payload attributes to notify EL of the new head.
    req := ctypes.BuildForkchoiceUpdateRequestNoAttrs(
        &engineprimitives.ForkchoiceStateV1{
            HeadBlockHash:      lph.GetBlockHash(),
            SafeBlockHash:      lph.GetParentHash(),
            FinalizedBlockHash: lph.GetParentHash(),
        },
        s.chainSpec.ActiveForkVersionForTimestamp(lph.GetTimestamp()),
    )
    if _, err = s.executionEngine.NotifyForkchoiceUpdate(ctx, req); err != nil {
        return fmt.Errorf("failed forkchoice update, head %s: %w",
            lph.GetBlockHash().String(),
            err,
        )
    }
    return nil
}
```

특징:

- 블록 최종화 후 실행 클라이언트에 새 헤드 통지
- Payload attributes 없이 전송 (옵티미스틱 빌드 요청 안 함)
- Safe/Finalized 해시는 부모 해시로 설정

7.2 Force Sync

```go
// beacon-kit/beacon/blockchain/payload.go (lines 77-130)
// 실제 forceSyncUponFinalize 구현 (일부)

// forceSyncUponFinalize sends a new payload and force startup FCU to the Execution
// Layer client. This informs the EL client of the new head and forces a SYNC
// if blocks are missing. This function should only be run once at startup.
func (s *Service) forceSyncUponFinalize(
    ctx context.Context,
    beaconBlock *ctypes.BeaconBlock,
    parentProposerPubkey *crypto.BLSPubkey,
) error {
    // NewPayload call first to load payload into EL client.
    executionPayload := beaconBlock.GetBody().GetExecutionPayload()
    payloadReq, err := ctypes.BuildNewPayloadRequestFromFork(beaconBlock, parentProposerPubkey)
    if err != nil {
        return err
    }

    if err = payloadReq.HasValidVersionedAndBlockHashes(); err != nil {
        return err
    }

    // We set retryOnSyncingStatus to false here. We can ignore SYNCING status and proceed
    // to the FCU.
    err = s.executionEngine.NotifyNewPayload(ctx, payloadReq, false)
    if err != nil {
        return fmt.Errorf("startSyncUponFinalize NotifyNewPayload failed: %w", err)
    }

    // Submit the forkchoice update to the EL client. This will ensure that it is either synced or
    // starts up a sync.
    req := ctypes.BuildForkchoiceUpdateRequestNoAttrs(
        &engineprimitives.ForkchoiceStateV1{
            HeadBlockHash:      executionPayload.GetBlockHash(),
            SafeBlockHash:      executionPayload.GetParentHash(),
            FinalizedBlockHash: executionPayload.GetParentHash(),
        },
        s.chainSpec.ActiveForkVersionForTimestamp(executionPayload.GetTimestamp()),
    )

    switch _, err = s.executionEngine.NotifyForkchoiceUpdate(ctx, req); {
    case err == nil:
        return nil

    case errors.IsAny(err,
        engineerrors.ErrSyncingPayloadStatus,
        engineerrors.ErrAcceptedPayloadStatus):
        s.logger.Warn(
            `Your execution client is syncing. It should be downloading eth blocks from its peers. Restart the beacon node once the exec
        )
        return err

    default:
        return fmt.Errorf("force startup NotifyForkchoiceUpdate failed: %w", err)
    }
}
```

시작 시 동기화:

1. NewPayload로 페이로드 로드
2. ForkchoiceUpdate로 동기화 강제
3. SYNCING 상태 처리
4. 동기화 완료 대기 안내

7.3 Payload 빌드 데이터 준비

```go
// beacon-kit/beacon/blockchain/payload.go (lines 136-150)
// 실제 preFetchBuildData 구현 (일부)

// preFetchBuildData prepares data needed to build the next block
func (s *Service) preFetchBuildData(st *statedb.StateDB, currentTime math.U64) (
    *builder.RequestPayloadData,
    error,
) {
    lph, err := st.GetLatestExecutionPayloadHeader()
    if err != nil {
        return nil, fmt.Errorf("failed retrieving latest execution payload header: %w", err)
    }
    nextPayloadTimestamp := payloadtime.Next(
        currentTime,
        lph.GetTimestamp(),
        true, // buildOptimistically
    )

    stateSlot, err := st.GetSlot()
    // ... 다음 블록 빌드 데이터 준비
```

빌드 준비:

- 최신 실행 페이로드 헤더 조회
- 다음 페이로드 타임스탬프 계산
- 옵티미스틱 빌드 활성화

---

8. 검증 로직

8.1 Stateless 검증

```go
// beacon-kit/state-transition/core/state_processor_payload.go (lines 120-138)
// 실제 validateStatelessPayload 구현

// validateStatelessPayload performs stateless checks on the execution payload.
func (sp *StateProcessor) validateStatelessPayload(blk *ctypes.BeaconBlock) error {
    body := blk.GetBody()
    payload := body.GetExecutionPayload()

    // Verify the number of withdrawals.
    withdrawals := payload.GetWithdrawals()
    if uint64(len(withdrawals)) > sp.cs.MaxWithdrawalsPerPayload() {
        return errors.Wrapf(
            ErrExceedMaximumWithdrawals,
            "too many withdrawals, expected: %d, got: %d",
            sp.cs.MaxWithdrawalsPerPayload(), len(withdrawals),
        )
    }

    // No need to verify bounded number of commitments here, since it is
    // verified early on in ProcessProposal.
    return nil
}
```

검증 항목:

- 출금(withdrawal) 개수 제한 검증
- 커밋먼트 개수는 ProcessProposal에서 검증됨

8.2 체인 스펙 활용

BeaconKit은 **ChainSpec**을 통해 네트워크 파라미터를 관리합니다:

```
type ChainSpec interface {
    MaxWithdrawalsPerPayload() uint64
    EpochsPerHistoricalVector() uint64
    ActiveForkVersionForTimestamp(timestamp math.U64) common.Version
    // ... 기타 스펙 메서드
}
```

지원 체인스펙:

- `mainnet` : 메인넷
- `testnet` : 테스트넷
- `devnet` : 개발 네트워크
- 커스텀: spec.toml 파일로 정의

---

9. Omni/MiniEVM과의 비교

9.1 아키텍처 비교표

항목	BeaconKit	Omni Octane	Initia MiniEVM
EVM 통합	Engine API (외부)	Engine API (외부)	내장 EVM
합의 메커니즘	CometBFT	CometBFT	CometBFT
실행 클라이언트	Bera-Geth/Reth	Geth 호환	자체 EVM
상태 관리	Beacon Chain 스타일	옵티미스틱 빌드	Collections
ABCI 버전	ABCI 2.0	ABCI++	ABCI 2.0
옵티미스틱 빌드	지원	지원	미지원
상태 캐싱	메모리 캐시	Payload 캐시	없음
BLS 서명	지원	미지원	미지원
포크 관리	타임스탬프 기반	버전 기반	버전 기반
출금 처리	Beacon 스타일	Engine API	EVM 내장

9.2 Engine API 사용 비교

BeaconKit

```go
// BeaconKit의 접근 방식:
// 1. Beacon Chain 스타일 상태 관리
// 2. BLS 서명 검증
// 3. 상태 캐싱으로 재실행 방지

func (sp *StateProcessor) validateStatefulPayload(...) error {
    // Beacon 상태 조회
    lph, err := st.GetLatestExecutionPayloadHeader()

    // 체인 정합성 검증
    if safeHash != payload.GetParentHash() {
        return ErrParentPayloadHashMismatch
    }

    // Engine API 호출
    sp.executionEngine.NotifyNewPayload(ctx, payloadReq, true)

    // RANDAO 검증 (Beacon 특유)
    if payload.GetPrevRandao() != expectedMix {
        return ErrRandaoMixMismatch
    }
}
```

Omni Octane

```go
// Octane의 접근 방식:
// 1. 옵티미스틱 빌드 중심
// 2. Payload 캐싱
// 3. 이벤트 프로세서

func (k *Keeper) PrepareProposal(...) (*abci.ResponsePrepareProposal, error) {
    // 옵티미스틱 빌드된 payload 가져오기
    payload := k.getOptimisticPayload(...)

    // Engine API로 payload 검증
    k.engineCl.NewPayloadV3(ctx, payload)

    // 이벤트 처리
    for _, proc := range k.eventProcs {
        proc.Process(payload.Logs)
    }
}
```

MiniEVM

```go
// MiniEVM의 접근 방식:
// 1. 내장 EVM 직접 실행
// 2. Collections 상태 관리
// 3. PreBlock 훅

func (k Keeper) Call(ctx context.Context, msg *types.MsgCall) (*types.MsgCallResponse, error) {
    // 직접 EVM 실행 (Engine API 없음)
    result, err := k.evm.Call(vmtypes.TxContext{
        From: from,
        To:   to,
        Data: msg.Data,
    })

    // Collections로 상태 저장
    k.VMStore.Set(ctx, key, value)
}
```

9.3 성능 특성 비교

측면	BeaconKit	Octane	MiniEVM
트랜잭션 처리	매우 빠름 (외부 EVM)	매우 빠름 (외부 EVM)	빠름 (내장)
상태 동기화	캐시 활용 (빠름)	옵티미스틱 (빠름)	직접 (보통)
메모리 사용	높음 (캐시)	중간 (payload)	낮음
네트워크 대역폭	중간 (RPC)	중간 (RPC)	낮음
검증 지연	낮음 (병렬)	낮음 (병렬)	중간

---

10. 성능 및 최적화

10.1 상태 캐싱 전략

BeaconKit은 **검증된 블록의 상태를 메모리에 캐싱**하여 재실행을 방지합니다:

```go
// consensus/cometbft/service/service.go
type Service struct {
    // cachedStates tracks in memory the post block states
    // of blocks which were successfully verified. It allows
    // finalizing without re-execution
    cachedStates cache.States
}

// FinalizeBlock에서 캐시 활용:
switch cached, err := s.cachedStates.GetCached(hash); {
case err == nil:
    // 캐시된 상태 사용 - 재실행 불필요
    finalState := cached.State
    return s.calculateFinalizeBlockResponse(req, cached.ValUpdates)
}
```

이점:

- ProcessProposal에서 검증한 블록을 FinalizeBlock에서 재실행 안 함
- 메모리 트레이드오프: 속도 향상 vs 메모리 사용 증가
- Genesis 블록 특수 처리로 초기화 안정성 보장

10.2 병렬 검증

```go
// state-transition/core/state_processor_payload.go
func (sp *StateProcessor) processExecutionPayload(...) error {
    var (
        g, ctx = errgroup.WithContext(txCtx.ConsensusCtx())
    )

    // 병렬 실행 1: Payload 검증
    if txCtx.VerifyPayload() {
        g.Go(func() error {
            return sp.validateExecutionPayload(...)
        })
    }

    // 병렬 실행 2: 헤더 생성
    g.Go(func() error {
        var err error
        header, err = payload.ToHeader()
        return err
    })

    // 모든 고루틴 완료 대기
    if err := g.Wait(); err != nil {
        return err
    }
}
```

최적화:

- `errgroup` 으로 검증과 헤더 생성 병렬화
- 컨텍스트 전파로 에러 시 즉시 취소
- 타임아웃 관리

10.3 메트릭 수집

```go
// execution/client/engine.go
func (s *EngineClient) NewPayload(...) (*common.ExecutionHash, error) {
    var (
        startTime    = time.Now()
        cctx, cancel = s.createContextWithTimeout(ctx)
    )
    defer s.metrics.measureNewPayloadDuration(startTime)
    defer cancel()

    // ... 실행 ...

    if errors.Is(err, engineerrors.ErrEngineAPITimeout) {
        s.metrics.incrementNewPayloadTimeout()
    }
}
```

추적 메트릭:

- NewPayload 지연 시간
- ForkchoiceUpdate 지연 시간
- GetPayload 지연 시간
- 타임아웃 발생 횟수
- 가스 사용량

10.4 타임아웃 관리

```
// execution/client/client.go (실제 코드 기반)
// Enforcing minimum rpc timeout
// The reason we do it is that we previously suggested a
// 900 ms default, which is unnecessarily strict.
if cfg.RPCTimeout < MinRPCTimeout {
    logger.Warn("Automatically raising RPCTimeout",
        "configured", cfg.RPCTimeout,
        "minimum", MinRPCTimeout,
    )
}
cfg.RPCTimeout = max(MinRPCTimeout, cfg.RPCTimeout)
```

타임아웃 정책:

- 최소 RPC 타임아웃 강제
- 경고 로그로 설정 안내
- 자동 조정

---

11. 장단점 분석

11.1 BeaconKit의 장점

1. Ethereum 호환성

```
// Engine API 표준 준수로 모든 Ethereum 실행 클라이언트 지원
// - Geth
// - Reth
// - Nethermind
// - Besu
// - Erigon
```

이점:

- Ethereum 생태계 도구 직접 사용 가능
- 업그레이드 자동 추적 (Shanghai, Cancun, Deneb, Electra 등)
- 실행 레이어 최적화를 합의 레이어와 독립적으로 진행

2. 모듈러 아키텍처

```
합의 레이어 (BeaconKit)
    ↕ Engine API
실행 레이어 (Geth/Reth)
```

이점:

- 각 레이어 독립 개발/배포
- 실행 클라이언트 교체 용이
- 장애 격리

3. 상태 캐싱

```
// 검증된 블록 재실행 불필요
cachedStates.GetCached(hash) // O(1) 조회
```

이점:

- FinalizeBlock 속도 향상
- 네트워크 처리량 증가
- 검증자 리소스 절약

4. Beacon Chain 보안 모델

```
// BLS 서명, RANDAO, 출금 등 Ethereum Beacon Chain 검증 로직 활용
if payload.GetPrevRandao() != expectedMix {
    return ErrRandaoMixMismatch
}
```

이점:

- 검증된 보안 모델
- Ethereum 커뮤니티 감사 혜택
- 장기 지속성

11.2 BeaconKit의 단점

1. 복잡도

```
BeaconKit 스택:
- CometBFT
- Beacon Chain 로직
- Engine API
- Execution Client
- 상태 캐싱
- BLS 서명
```

단점:

- 학습 곡선 높음
- 디버깅 어려움
- 운영 오버헤드

2. 외부 의존성

```
// 실행 클라이언트 필수
executionEngine ExecutionEngine
```

단점:

- 두 프로세스 관리 필요
- RPC 장애 가능성
- 네트워크 지연

3. 메모리 사용

```
// 상태 캐싱으로 메모리 소비
cachedStates cache.States
```

단점:

- 높은 메모리 요구사항
- 캐시 무효화 오버헤드
- OOM 위험

4. Cosmos SDK 통합 제한

```
// Cosmos 모듈을 EVM과 통합하기 어려움
// Precompile이나 별도 메커니즘 필요
```

단점:

- Cosmos 생태계 도구 제한적
- 커스텀 로직 추가 복잡
- MiniEVM보다 유연성 낮음

---

12. 실전 예제

12.1 BeaconKit 노드 시작

```
# 1. Consensus 클라이언트 시작
make start

# 2. Execution 클라이언트 시작 (별도 터미널)
make start-reth  # 또는 make start-geth

# 3. 연결 확인
beacond status
```

12.2 커스텀 ChainSpec 사용

```toml
# spec.toml
[chain]
chain_id = 12345
network_name = "my-custom-chain"

[genesis]
genesis_time = 1704067200

[consensus]
slot_duration = "2s"
epochs_per_historical_vector = 65536

[execution]
max_withdrawals_per_payload = 16
```

```
beacond init --beacon-kit.chain-spec=custom --beacon-kit.chain-spec-file=./spec.toml
beacond start
```

12.3 상태 캐싱 비활성화 (디버깅)

```go
// consensus/cometbft/service/finalize_block.go 수정
// 디버깅 시 캐시 비활성화

func (s *Service) finalizeBlock(...) (*cmtabci.FinalizeBlockResponse, error) {
    // 캐시 조회 건너뛰기
    // switch cached, err := s.cachedStates.GetCached(hash); {
    // ...
    // }

    // 항상 재실행
    finalState := s.resetState(ctx)
    valUpdates, err := s.Blockchain.FinalizeBlock(finalState.Context(), req)
    // ...
}
```

12.4 커스텀 실행 클라이언트 연결

```toml
# app.toml
[engine]
rpc-dial-url = "http://my-custom-geth:8551"
rpc-timeout = "5s"
rpc-jwt-refresh-interval = "1m"
```

```go
// JWT 시크릿 생성
jwtSecret := jwt.NewRandom()

// EngineClient 생성
engineClient := client.New(
    cfg,
    logger,
    jwtSecret,
    telemetrySink,
    eth1ChainID,
)

// 시작
engineClient.Start(ctx)
```

12.5 Payload 검증 로직 확장

```go
// 커스텀 검증 로직 추가
type CustomStateProcessor struct {
    *core.StateProcessor
    customValidator CustomValidator
}

func (csp *CustomStateProcessor) validateStatefulPayload(...) error {
    // 기본 검증
    if err := csp.StateProcessor.validateStatefulPayload(...); err != nil {
        return err
    }

    // 커스텀 검증 추가
    if err := csp.customValidator.Validate(payload); err != nil {
        return fmt.Errorf("custom validation failed: %w", err)
    }

    return nil
}
```

12.6 메트릭 확인

```go
// Prometheus 메트릭 엔드포인트
// http://localhost:26660/metrics

// 주요 메트릭:
// - beacon_kit_runtime_prepare_proposal_duration
// - beacon_kit_engine_new_payload_duration
// - beacon_kit_engine_forkchoice_update_duration
// - beacon_kit_engine_get_payload_duration
// - beacon_kit_block_gas_used
```

요약

BeaconKit의 핵심 특징

1. **Ethereum 호환**: Engine API로 표준 Ethereum 실행 클라이언트 지원
2. **Beacon Chain 모델**: 검증된 보안 및 상태 전이 로직
3. **CometBFT 합의**: 빠른 최종성과 Byzantine 장애 허용
4. **상태 캐싱**: 재실행 방지로 성능 향상
5. **모듈러 설계**: 합의와 실행 레이어 분리

Omni/MiniEVM과의 차이

측면	BeaconKit	Octane	MiniEVM
철학	Ethereum 표준 준수	옵티미스틱 최적화	Cosmos 통합
복잡도	높음	중간	낮음
성능	매우 높음	매우 높음	높음
유연성	낮음	중간	높음
생태계	Ethereum	Omni	Initia

사용 권장 사항

BeaconKit 적합:

- Ethereum 메인넷과 최대 호환성 필요
- Beacon Chain 보안 모델 선호
- 실행 클라이언트 교체 유연성 필요
- 높은 처리량과 낮은 지연 시간 요구

BeaconKit 부적합:

- Cosmos SDK 모듈과 깊은 통합 필요
- 단일 프로세스 배포 선호
- 메모리 제약 환경
- 간단한 아키텍처 선호

다음 단계: Phase 2의 다음 문서에서는 이 세 가지 EVM 통합 방식(BeaconKit, Octane, MiniEVM)의 종합 비교와 선택 가이드를 제공합니다.